

Table 3. Illustrating entity resolution via a qualitative example. The expressive generations show that as we go through the layers, more tokens from the context get integrated into the current representation. The “Generation” column shows the automatically generated text. The “Explanation” column shows our own manually coded interpretation, aiming to specify what entity the generation refers to and how that relates to the tokens processed. $\mathcal{M}^* = \mathcal{M} \leftarrow \text{Vicuna (13B)}$, $\ell^* = \ell$, $S \leftarrow \text{"Diana, Princess of Wales"}$.

ℓ	Generation	Explanation
1-2	: Country in the United Kingdom	Wales
3	: Country in Europe	Wales
4	: Title held by female sovereigns in their own right or by queens consort	Princess of Wales (unspecific)
5	: Title given to the wife of the Prince of Wales (and later King)	Princess of Wales (unspecific)
6	: Diana, Princess of Wales (1961-1997), the first wife of Prince Charles, Prince of Wales, who was famous for her beauty and humanitarian work	Diana, Princess of Wales

contextualization is performed.

We analyze how LLMs contextualize input entity names by leveraging Patchscopes. Particularly, we craft a target prompt for generating a description of a given subject, and apply it to the hidden representation at the last subject position in the source prompt – where the model forms the subject representation (Geva et al., 2023; Hernandez et al., 2023a) – across the early layers. This will allow us to see how the model describes the subject in each layer.

Analysis Setting We use a few-shot target prompt template for decoding an entity description: “subject₁: description₁, ..., subject_k: description_k, x”, while patching the last position corresponding to x. We take the 200 most popular and 200 least popular subject entities from the PopQA dataset (Mallen et al., 2023). The popular entities should appear frequently in LLMs’ pre-training data, and are thus likely to be captured by the model, while resolving the rare entities is expected to be more challenging (Kandpal et al., 2023; Mallen et al., 2023). Then, for the source prompt we use the entity name, and for the target prompt we sample $k = 3$ random subject entities. We obtain a short (up to one sentence) description of every subject entity from Wikipedia. Our target prompt and more technical details are provided in §D.1. We patch the last position representations from the first 10 layers of Vicuna 13B to the target prompt and evaluate the generated subject name and description. Specifically, the generated descriptions are evaluated against the descriptions from Wikipedia using RougeL (Lin, 2004). Evaluation with Rouge1 (Lin, 2004) and Sentence-Bert (Reimers & Gurevych, 2019) shows

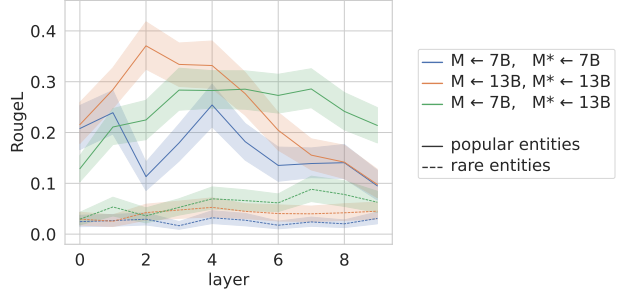


Figure 3. RougeL scores of the generated descriptions against descriptions from Wikipedia, using Vicuna models.

similar trends (see §D.2).

Results Tab. 3 illustrates the generations by Vicuna 13B for a sample subject entity, when patching its representation at different layers to the target prompt (see more examples in §D.3). For most entities, the contextualization process is spread over the first layers, with the last subject token gradually encompassing more distant positions across layers.

This trend can be quantitatively observed by the similarity between the generated descriptions and the descriptions from Wikipedia, as measured by RougeL. See Fig. 3 where $\mathcal{M} = \mathcal{M}^*$. For both models, similarity increases in the first 5 layers and then slowly decreases. This decrease could potentially be attributed to contamination caused by the representation of the placeholder token “x” remaining in the early layers, when patching is applied to a later layer. Note that this potential issue is only applicable to multi-token generation scenarios as future positions can still attend to the placeholder position in early layers, potentially interfering with the model’s ability to accurately generate descriptions for the patched token. See §D.3 for qualitative examples corroborating this interpretation. As expected, the scores for rare, long-tail entities are significantly lower than those of popular entities. See §D.2 for additional results with Pythia where the smaller model seems to outperform the larger model, possibly because the larger model is biased toward output generation at the expense of input contextualization. To summarize, this analysis shows Patchscopes’ utility for inspecting the contextualization process in early layers.

4.4. Expressiveness from Cross-Model Patching

A possible avenue for improving inspection capabilities is to explain a given model with a model that is more expressive (Bills et al., 2023). In Patchscopes, this means to patch a representation of $\mathcal{M}$ into a more expressive model $\mathcal{M}^*$. However, it is not clear if such an intervention would yield plausible results, due to possible discrepancies between the two models resulting from different architectures, optimization processes, and so on. Here, we present experiments on next-token prediction and entity resolution that exemplify

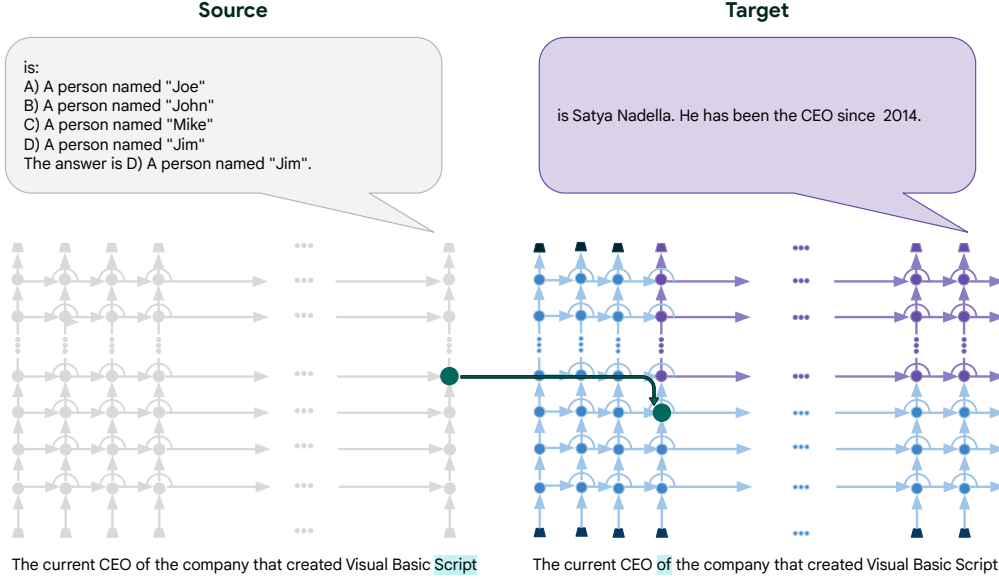


Figure 4. An illustration of CoT Patchscope on a single example. In this example, $\pi_1 \leftarrow$ “the company that created Visual Basic Script”, $\pi_2 \leftarrow$ “The current CEO of”, $S = T \leftarrow [\pi_2][\pi_1] =$ “The current CEO of the company that created Visual Basic Script”. Note that $\mathcal{M} = \mathcal{M}^*$ and $f \leftarrow \mathbb{I}$.

not only the feasibility, but also the opportunities unlocked by patching across models of the same family.

Next-Token Prediction We repeat the experiment in §4.1, using the token identity Patchscope. To overcome discrepancies between the models, we learn affine mappings between their layers (similarly to Tuned Lens). Our results show that source and target layers on the diagonal exhibit the highest precision values, and that patching representations to an early layer of the larger model are the most effective. Overall, suggesting that when $\mathcal{M}^*$ and $\mathcal{M}$ are from the same model family, it is possible to leverage $\mathcal{M}^*$ for decoding information from the representations of $\mathcal{M}$. For detailed results on Precision@1 and Surprisal, see §E.

Entity Resolution in Early Layers We now show that using a large model as $\mathcal{M}^*$ can enhance the output expressivity. To this end, we repeat our entity resolution experiment in §4.3 with Vicuna model family, setting $\mathcal{M} \leftarrow 7B$, $\mathcal{M}^* \leftarrow 13B$. Fig. 3 shows the cross-model patching results (green lines) compared to the same-model patching $\mathcal{M} \leftarrow 7B$, $\mathcal{M}^* \leftarrow 7B$ (blue line). The results show that cross-model patching from a smaller model to its larger version generally improves the ability to inspect the input contextualization, both for popular and rare entities. For Pythia, since the smaller model outperforms the larger one, cross-model patching is not as effective (see §D.2).

5. Application: Correcting Multi-Hop Errors

Multi-hop reasoning is a challenging problem (Zhong et al., 2023). While a language model may be capable of correctly answering each step independently, it could still fail at processing the connection between different steps, resulting in an incorrect prediction. Recent attempts to improve multi-hop reasoning rely on prompting the model to generate a step-by-step answer autoregressively (e.g., Wei et al., 2022; Yao et al., 2023; Besta et al., 2024), some with an iterative process of self-refinement (e.g., Madaan et al., 2023). However, achieving similar benefits might be possible via directly rewiring the model’s intermediate computation.

Here, we show that Patchscopes can improve multi-hop reasoning performance *without* generating the reasoning steps, particularly in cases where the model fails at completing a multi-hop query despite being successful in each reasoning step independently. Via Patchscopes, one can surgically operate on the model representations, reroute its intermediate answer to one reasoning step, simplify the consequent step, and ultimately correct the final prediction.

Data Building on Hernandez et al. (2023b), we systematically generate all valid multi-hop factual and commonsense reasoning queries where $\omega_1 = \sigma_2$. We conduct experiments on Vicuna (13B), focusing on samples where $\mathcal{M}$ accurately represents both τ_1 and τ_2 independently, that is, ω appears in the next 20 tokens $\mathcal{M}$ generates conditioned on the prompt π that verbalizes σ and ρ . This process yields 1,104 multi-

hop reasoning samples, out of which 46 satisfy the above criteria and are used for evaluation. See more details in §F.

Experimental Setup Following the notation in §4.2, let $\tau_1 = (\sigma_1, \rho_1, \omega_1)$ represent the relation ρ_1 between a subject entity σ_1 and an object entity ω_1 . Let $\tau_2 = (\sigma_2, \rho_2, \omega_2)$ represent another tuple such that $\sigma_2 = \omega_1$. A multi-hop reasoning query pertaining to τ_1 and τ_2 is a prompt composed of two parts: π_1 is a verbalization of σ_1 and ρ_1 from which ω_1 can be inferred; π_2 is a verbalization of ρ_2 , from which ω_2 can be inferred after its concatenation with π_1 . For example, Let $\tau_1 \leftarrow$ (“Visual Basic”, “product of”, “Microsoft”) and $\tau_2 \leftarrow$ (“Microsoft”, “company CEO”, “Satya Nadella”). An example verbalization of these tuples is $\pi_1 \leftarrow$ “the company that created Visual Basic”, $\pi_2 \leftarrow$ “The current CEO of”, leading to the multi-hop query $[\pi_2][\pi_1] =$ “The current CEO of the company that created Visual Basic”.

Method We introduce a Chain-of-Thought (CoT) Patchscope to fix multi-hop reasoning via intervening on the computation graph and rerouting representation likely to capture ω_1 in place of σ_2 . Concretely, S refers to the formed query discussed above, and we use the following configuration: $T \leftarrow S, \mathcal{M}^* \leftarrow \mathcal{M}, i \leftarrow n, i^* \leftarrow$ the token preceding π_1 . We evaluate the outputs in terms of accuracy, similarly to §4.2. For a sample S , the Patchscope is considered accurate if $\exists (\ell, \ell^*) : \ell \in [1, \dots, L], \ell^* \in [1, \dots, L^*]$ where the autoregressive generation up to 20 tokens includes ω_2 . Fig. 4 illustrates the CoT Patchscope with an example. We use the following configuration for CoT Patchscope: $S \leftarrow \pi_1, T \leftarrow \pi_2, i \leftarrow n, i^* \leftarrow m$, which is equivalent to $S = T \leftarrow [\pi_2][\pi_1]$ and adjusting the attention mask such that no token in S has visibility to π_2 and no token in T has visibility to π_1 . In addition, we consider two baselines.

Vanilla Baseline For this baseline, we set $S \leftarrow [\pi_1][\pi_2]$, we let the model autoregressively generate up to 20 tokens and check whether ω_2 appears in the generation.

Chain-of-Thought Baseline Here, the setup and evaluation is similar to the vanilla baseline, except that we prepend “Let’s think step by step.” to S , following (Wei et al., 2022). We then let the model generate up to 20 tokens and check whether ω_2 appears in the generation. Note that this experiment uses Vicuna (13B). Vicuna is based on LLaMA, with supervised finetuning on additional instruction data, which makes it amenable to chain-of-thought prompting.

Results While the vanilla baseline accuracy is only 19.57%, and CoT baseline accuracy is 35.71%, our proposed Patchscope achieves 50% accuracy. For more

details about the interaction between ℓ and ℓ^* , and how it affects the success rate, see Fig. 12 in §F.

We emphasize that our primary goal in this section is not to devise a new method to solve multi-hop queries that is necessarily a competitor to CoT, but rather to make a proof-of-concept while comparing with CoT as a common reference. We highlight that we have taken advantage of the extra structural information about the queries given the prior knowledge about how they were synthesized. One could potentially automate this by learning the right places to patch. Li et al. (2024) recently proposed an optimization-based approach for directly patching multi-head self-attention in mid-layers, which can be viewed as soft position-selection, and works effectively in multihop error correction, suggesting that inferring the right positions to patch could be effective. However, even if optimal source and target position are automatically decided upfront, Patchscope and CoT may not be directly comparable. CoT generates multiple steps which fundamentally extend the computational power of the LLM (Merrill & Sabharwal, 2024), but a Patchscope with pre-identified (l, l^*) only makes $O(1)$ inference passes.

6. Conclusion

We present Patchscopes, a simple and effective framework that leverages the ability of LLMs to generate human-like text for decoding information from intermediate LLM representations. We show that many existing interpretability methods can be cast as specific Patchscope instances, and even these only cover a small portion of the framework’s possible configurations. Moreover, new underexplored Patchscopes substantially improve our ability to decode various types of information from the model’s internal computation, such as the output prediction and knowledge attributes, typically outperforming prominent methods that rely on projection to the vocabulary and probing. In addition, our framework enables new capabilities, such as analyzing the contextualization process of input tokens in early LLM layers, and can correct multi-hop reasoning.

There are multiple future research directions to consider. An important factor in the effectiveness of a chosen target prompt is how the information from the patched position propagates during inference to other positions and across layers. Understanding how to best use a given target prompt, perhaps automatically, is an important factor for using Patchscopes. Another avenue for future work is investigating the effectiveness of few-shot target prompts compared to zero-shot/instruction-based prompts. More expressive instruction-based target prompts, for example, could enable extracting more complex information. Additionally, while our cross-model patching focused on models from the same family, it will be valuable to explore which mapping functions would enable patching across models